

THINGS SENIOR DevOps ENGINEERS SHOULD KNOW



✓ REAL-WORLD
CONCEPTS

✓ PRACTICAL
EXAMPLES

✓ PRODUCTION
MINDSET

✓ SENIOR LEVEL
INSIGHTS

VERIQTA

ENGINEERS TODAY. A BETTER TOMORROW.

20
PAGES

Think in Systems, Not Tools

Senior DevOps engineers understand how everything connects.
Tools are only part of the bigger system.

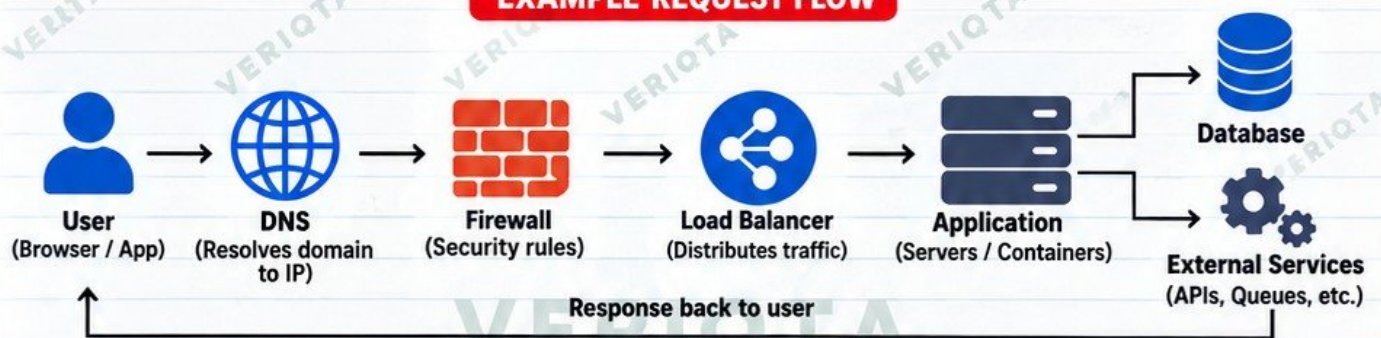
1. Understand the Complete Production System

- A production system is more than just an application.
- It includes users, network, DNS, load balancers, servers, containers, databases, external services, monitoring, and more.
- Understand how all the parts work together.

2. Request Path from User to Application

- Know how a request flows from the user to the application and back.
- Identify each component in the path.
- Understand where failures can occur.
- Be able to trace a request end to end.

EXAMPLE REQUEST FLOW



3. Dependencies and Failure Domains

- Every component depends on other components.
- Understand direct and indirect dependencies.
- Know the failure domain for each component.
- A failure in one area can affect multiple services.
- Map dependencies and their impact.

4. Control Plane vs Data Plane

- Control plane manages the system (e.g. Kubernetes API server, etcd, controllers).
- Data plane runs your actual workloads (e.g. application Pods, containers, traffic.).
- Understand the difference, because they fail differently and require different troubleshooting approaches.

5. Stateful vs Stateless Components

- Stateless components do not keep data between requests (e.g. web servers, API servers).
- Stateful components keep data (e.g. databases, caches, message queues).
- Stateful components usually require special care for backups, scaling, and failover.

6. Upstream and Downstream Dependencies

- Upstream means the services your application depends on.
- Downstream means the services that depend on your service.
- Know both, so you understand the full impact of a change or failure.
- Communicate with teams when something will affect upstream or downstream services.

7. Follow a Request End to End

- Use logs, metrics, and traces to follow a request.
- Identify where latency or errors occur.
- Practice tracing real requests in your environment.
- This skill is critical during incidents.

8. Understand Blast Radius Before Making Changes

- Always ask: What services, users, or regions will be affected?
- Consider dependencies, shared resources, and data.
- Make changes safely: test, use smaller rollouts, and have a rollback plan.



9. Senior Mindset: Ask What Else Will Be Affected?

A senior engineer does not just make the change. They think ahead. Consider the full impact, communicate clearly, and plan for failure.

"It's not just about what you change.
It's about what that change affects."



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Linux Beyond Basic Commands

Senior DevOps engineers go beyond basic commands. They understand how Linux works under the hood, so they can troubleshoot, optimize, and keep systems reliable.

1. Processes, Signals, Threads, and Process Trees

- A process is a running program.
- A process can have multiple threads.
- Processes form a tree (parent and child).
- Signals are used to control processes (e.g. SIGTERM, SIGKILL).
- Understand how to find, manage, and troubleshoot processes.

Useful commands:

`ps aux` `pstree` `top` `kill` `pkill`

2. CPU, Memory, Load Average, and Pressure

- Monitor CPU usage, memory usage, and system load.
- Load average shows the number of processes waiting for CPU.
- Identify resource pressure before the system becomes unstable.
- Use real-time tools to find what is consuming resources.

Useful commands:

`top` `htop` `vmstat` `free`
`uptime` `sar`

3. File Descriptors and Open Files

- Every process uses file descriptors for files, network connections, and other resources.
- Too many open files can cause application failures.
- Check limits and open files for a process.

Useful commands:

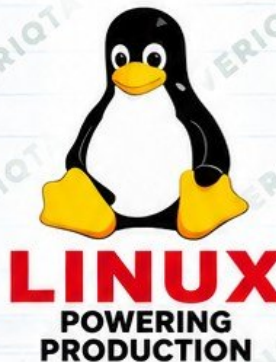
`lsof` `lsof -p <PID>`
`ulimit -n` `cat /proc/<PID>/limits`

4. Filesystems, Mounts, Inodes, and Disk Pressure

- Understand filesystem types (ext4, xfs, etc).
- Check disk usage, inode usage, and mounts.
- Disk full or inode full can break applications.
- Know how to identify what is using space.

Useful commands:

`df -h` `du -sh` `df -i` `mount`
`findmnt` `lsblk`



5. systemd and Service Management

- systemd manages services, processes, and system startup.
- Start, stop, restart, enable, and check service status.
- View service logs with journalctl.

Useful commands:

`systemctl status <service>`
`systemctl start <service>`
`systemctl enable <service>`
`journalctl -u <service> -f`

6. Kernel and System Logs

- Kernel logs show low-level system events (drivers, hardware, kernel messages).
- System logs help you troubleshoot service and application issues.
- Learn where to find and read logs.

Useful commands:

`dmesg` `journalctl` `journalctl -k`
`tail -f /var/log/syslog`

7. Permissions, Ownership, Capabilities, and sudo

- Files and directories have owners, groups, and permissions.
- Understand special permissions and Linux capabilities.
- Use sudo safely and avoid giving unnecessary root access.

Useful commands:

`ls -l` `chmod` `chown` `chgrp`
`getcap` `sudo -l`

8. Key Commands Every Engineer Uses

Command	What It Does
<code>top</code>	Real-time system processes
<code>ps</code>	View running processes
<code>ss</code>	Check network connections
<code>lsof</code>	List open files
<code>df</code>	Check disk usage
<code>du</code>	Find directory size
<code>journalctl</code>	View system and service logs
<code>dmesg</code>	View kernel messages

9. Diagnose a Sick Linux Host Systematically

- 1 Identify the symptom (slow, high CPU, no disk space, service down, etc).
- 2 Check system resources (CPU, memory, disk, load).
- 3 Look at running processes and services.
- 4 Check logs (system, application, kernel).
- 5 Identify recent changes (deployments, config, packages).
- 6 Find the root cause.
- 7 Apply the fix.
- 8 Verify the system is healthy.

Senior Engineer Mindset

- ✓ Look at the whole system, not just one error.
- ✓ Correlate logs, metrics, and system state.
- ✓ Think about what changed.
- ✓ Understand the impact before making changes.
- ✓ Solve the root cause, not just the symptom.
- ✓ Keep systems secure, stable, and observable.



Remember This

Linux is the foundation of DevOps. The better you understand it, the faster you can troubleshoot, automate, and keep production systems running.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA

ENGINEERS TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW

Networking

You Must Actually Understand

SAME
NETWORK
BETTER
ENGINEERS

Senior DevOps engineers don't just use the network. They understand how it works, so they can design, troubleshoot, and keep systems running.

1. TCP/IP Fundamentals

- TCP/IP is the foundation of modern networking.
- IP handles addressing and routing.
- TCP provides reliable, ordered delivery.
- UDP is faster but not reliable.
- Understand how data is packetized and transported.

Useful commands:

`ip a ip r ping`

2. DNS Resolution

- DNS converts domain names to IP addresses.
- Understand recursive vs iterative queries.
- Know about DNS records (A, AAAA, CNAME, MX, etc).
- DNS issues can break entire applications.

Useful commands:

`dig nslookup host`

3. TCP Connection Lifecycle

- Understand the 3-way handshake (SYN, SYN-ACK, ACK).
- Know the connection states (ESTABLISHED, TIME_WAIT, etc).
- Understand how TCP handles retransmissions and flow control.

Useful commands:

`ss -t netstat tcpdump`

4. TLS and Certificates

- TLS encrypts data in transit.
- Understand the TLS handshake process.
- Know about certificates, CAs, and trust chains.
- Be able to check certificate details and expiry.
- TLS issues often cause connectivity failures.

Useful commands:

`openssl s_client curl -v`

5. Routing and Route Tables

- Routing determines where packets go.
- Understand route tables and next hops.
- Know the difference between local, static, and dynamic routes.
- Be able to troubleshoot routing issues.

Useful commands:

`ip route route -n`

6. NAT (Network Address Translation)

- NAT allows private IPs to access external networks.
- Understand SNAT, DNAT, and PAT.
- Common in cloud environments and home networks.
- Know how NAT can affect troubleshooting.

Useful commands:

`iptables -t nat -L ip route curl ifconfig.me`

7. Firewalls and Security Groups

- Firewalls control network traffic.
- Cloud security groups act as virtual firewalls.
- Allow or deny traffic based on IP, port, and protocol.
- Misconfigured rules are a common cause of outages.

Useful commands:

`iptables -L ufw status cloud CLI (e.g. aws ec2)`

8. Proxies and Reverse Proxies

- Proxies sit between clients and servers.
- Reverse proxies (e.g. Nginx) route traffic to backend services.
- Used for load balancing, security, and TLS termination.
- Understand when and why to use them.

Useful commands:

`curl -I nginx -t`

9. Load Balancing

- Distributes traffic across multiple servers.
- Improves availability and scalability.
- Understand different types (L4 vs L7).
- Know health checks and session persistence.

Useful commands:

`curl -I dig cloud CLI`

10. CIDR and Subnetting

- CIDR defines IP address ranges (e.g. 192.168.1.0/24).
- Understand subnets, network and broadcast addresses.
- Used in VPCs, security groups, and routing.
- Helps with IP planning and troubleshooting.

Useful examples:

`192.168.1.0/24 = 256 IPs`
`10.0.0.0/16 = 65,536 IPs`

11. MTU and Packet Fragmentation

- MTU is the maximum packet size (usually 1500 bytes).
- Packets larger than the MTU are fragmented.
- Fragmentation can cause connectivity and performance issues.
- Important in VPNs, cloud networks, and containers.

Useful commands:

`ping -M do -s 1472 <IP>`
`tracert <host>`

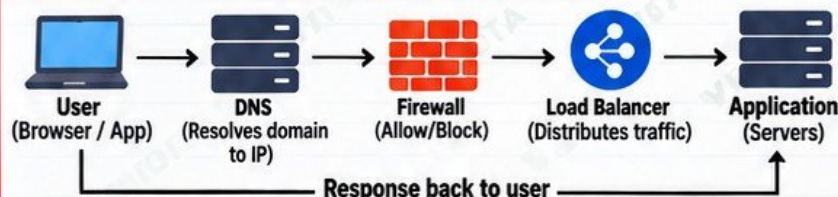
12. Common Network Failures and How to Diagnose

- DNS failures (cannot resolve domain).
- Connection timeouts (no response).
- Connection refused (service not listening).
- TLS failures (certificate or handshake issues).
- Use a systematic approach to find the root cause.

Useful commands:

`dig curl -v ss tcpdump`

EXAMPLE: REQUEST FLOW ACROSS A NETWORK



SENIOR ENGINEER MINDSET

- Understand the full path, not just one component.
- Always ask: What else could be affected?
- Use evidence, not assumptions.
- Check DNS, network, security, and application layers.
- Think about failure domains and blast radius.
- Build systems that are observable and debuggable.



KEY TAKEAWAY

The network is the invisible backbone of everything you run. Understanding it deeply makes you a stronger DevOps engineer.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA

ENGINEERS TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW

Cloud Architecture and Failure Domains

BETTER
ARCHITECTURE
SAFER
SYSTEMS
STRONGER
ENGINEERS

Senior DevOps engineers design for failure. They understand cloud architecture, so they can build systems that are reliable, scalable, and cost-effective.

1. Regions and Availability Zones

- A region is a separate geographic location (e.g. us-east-1).
- An availability zone (AZ) is an isolated data center within a region.
- Each AZ has independent power, network, and cooling.
- Use multiple AZs to improve availability.

Example (AWS):

Region: us-east-1

AZs: us-east-1a, us-east-1b, us-east-1c

2. High Availability vs Fault Tolerance

- High availability (HA) keeps your system running with minimal downtime.
- Fault tolerance (FT) keeps your system running even when a component fails.
- HA reduces downtime. FT eliminates single points of failure.

3. Horizontal vs Vertical Scaling

- Vertical scaling: increase CPU, memory, or storage on a single server.
- Horizontal scaling: add more servers and distribute traffic.
- Horizontal scaling is more resilient and commonly used in the cloud.

4. Load Balancing

- Distributes traffic across multiple servers.
- Improves availability and performance.
- Can be regional or global (e.g. AWS ALB, NLB, CloudFront, GCP LB).
- Performs health checks and removes unhealthy targets.

5. Autoscaling

- Automatically increases or decreases the number of instances based on demand.
- Helps handle traffic spikes.
- Reduces costs during low traffic.
- Uses metrics like CPU, memory, or custom metrics.

6. Stateless Architecture

- Stateless services do not store data locally.
- Any instance can handle any request.
- Easier to scale and recover from failures.
- Store data in external services (databases, object storage, caches).

7. Managed Services

- Cloud providers manage the infrastructure for you (e.g. RDS, EKS, S3, DynamoDB).
- Reduces operational work and improves reliability.
- You are still responsible for configuration, security, and monitoring.

8. Multi-AZ Design

- Deploy resources across multiple AZs in the same region.
- If one AZ fails, traffic continues to healthy AZs.
- Common for web apps, databases, and critical services.

9. Multi-Region Tradeoffs

- Improves disaster recovery and global availability.
- Increases cost, complexity, and operational overhead.
- Consider data replication, latency, and compliance requirements.
- Use when your business requires high resilience or global reach.

10. Single Points of Failure

- A single component whose failure can take down the system.
- Examples: single server, single database, single AZ, single load balancer.
- Identify and remove single points of failure.
- Use redundancy and failover mechanisms.

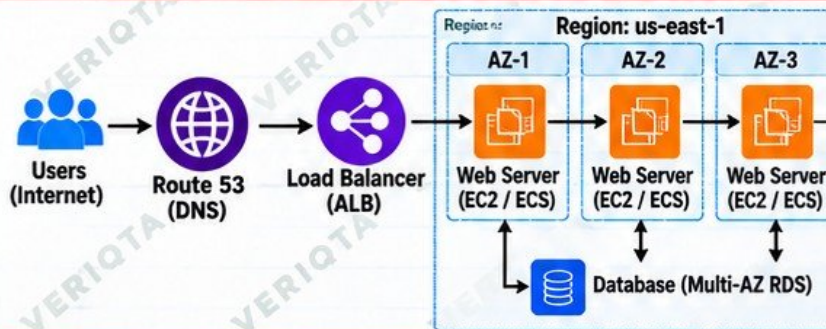
11. Cost, Complexity, Reliability, and Security Tradeoffs

- More reliability usually increases cost and complexity.
- More security may require additional tools and processes.
- Managed services can reduce complexity but may increase cost.
- Senior engineers choose the right balance based on business needs, not just technology.

12. Design for Component Failure

- Assume any component can and will fail.
- Use redundancy, health checks, retries, and graceful degradation.
- Test failure scenarios (e.g. AZ outage, database failure).
- Build systems that recover automatically.
- Have a rollback and recovery plan.

EXAMPLE: HIGHLY AVAILABLE CLOUD ARCHITECTURE (MULTI-AZ)



REAL-WORLD SCENARIO

If one AZ goes down, the load balancer routes traffic to healthy AZs and the application keeps running.



SENIOR ENGINEER MINDSET

- ✓ What happens if this component fails?
- ✓ Is there a single point of failure?
- ✓ How will this scale?
- ✓ What are the cost and security implications?
- ✓ Have we tested failure scenarios?
- ✓ Will the system recover automatically?



KEY TAKEAWAY

Good cloud architecture is not about using more services. It is about designing simple, reliable, secure, and cost-effective systems that can survive failure.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA

ENGINEERS TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW

SAME
INFRASTRUCTURE
BETTER
ENGINEERS

CODE
INFRASTRUCTURE
REPEAT
SCALE



Terraform

Infrastructure as Code at Scale

Senior DevOps engineers use Infrastructure as Code (IaC) to build, manage, and scale reliable, secure, and repeatable infrastructure.

1. Terraform Architecture

- Terraform uses providers to communicate with cloud platforms.
- You define desired infrastructure in configuration files (HCL).
- It creates an execution plan and applies changes through APIs.



2. State and Remote Backends

- State file keeps track of real infrastructure.
- Use remote backends (S3, Azure Storage, GCS) for team collaboration.
- Never use local state in production.

Backend Example (AWS S3)

```
terraform {
  backend "s3" {
    bucket = "veriqta-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
  }
}
```

3. State Locking

- Prevents multiple people from making changes at the same time.
- Avoids state corruption.
- Use DynamoDB (AWS), Blob Storage (Azure), or GCS for locking.



State Lock
= Safer Changes

4. Modules and Reusable Infra

- Modules help you reuse code and keep configurations clean.
- Use community or private modules.
- Pass variables to make modules flexible.

Module Example

```
module "vpc" {
  source = "terraform-aws-modules/vpc"
  version = "5.0.0"
  cidr = "10.0.0.0/16"
}
```

5. Environment Separation

- Use separate environments (dev, staging, prod).
- Keep state files and configurations separate.
- Use workspaces or separate folders.



6. Dependency Management

- Resources depend on each other.
- Use explicit and implicit dependencies.
- Understand when to use "depends_on".

Example

```
resource "aws_instance" "app" {
  ami      = "ami-123456"
  instance_type = "t3.micro"
  depends_on = [aws_vpc.main]
}
```

7. Provider / Version Management

- Use specific provider versions to avoid unexpected changes.
- Keep Terraform and provider versions consistent across teams.

Example

```
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

8. Drift

- Drift occurs when real infrastructure is different from your configuration.
- Can be caused by manual changes in the cloud console.
- Use "terraform plan" to detect drift and reconcile changes.



Detect Drift Early

9. Imports

- Use imports to bring existing infrastructure under Terraform management.
- Helps avoid recreating resources.

Example

```
terraform import \
aws_instance.web \
i-0abc123def456
```

10. Plan Review

- Always review the execution plan before applying.
- Check what will be created, changed, or destroyed.
- Look for unexpected changes.

\$ terraform plan

```
+ create
~ update
- destroy
```

11. CI/CD for Infrastructure

- Automate Terraform with CI/CD tools (GitHub Actions, GitLab CI, Jenkins, etc.).
- Run plan on pull requests.
- Require approvals for production.
- Use separate pipelines for different environments.



Automate for Consistency and Safety

12. Policy as Code

- Use tools like Sentinel, OPA (Open Policy Agent), or Terraform Cloud policies.
- Enforce security, cost, and compliance rules.
- Prevent risky changes before they are applied.



Policies Keep
Your Infrastructure Safe

13. Safe Infrastructure Changes

- Use small, incremental changes.
- Test in non-prod first.
- Have a rollback plan.
- Use code reviews and team approvals.
- Verify after apply.



Plan → Review → Apply → Verify

14. Recovering from Bad State or Partial Deployment

- Use state backups (remote backends).
- Fix state manually with "terraform state" commands when needed.
- Use "-target" carefully for partial deployments.
- If a deployment fails, understand what was created and clean up safely.
- Document and learn from the failure.

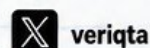
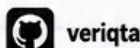
Common Recovery Commands

```
terraform state list
terraform state show <resource>
terraform state rm <resource>
terraform import <resource> <id>
```



KEY TAKEAWAY

Infrastructure as Code is not just about writing configuration. It is about building repeatable, secure, and reliable infrastructure at scale.



VERI QTA

ENGINEERS TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW



Kubernetes Beyond kubectl

Senior DevOps engineers understand how Kubernetes works, not just how to run kubectl. They can design, operate, troubleshoot, and scale real production clusters.

1. Kubernetes Control Plane

- The control plane manages the cluster.
- Key components: API server, etcd, scheduler, controller manager.
- It maintains the desired state and makes the actual state match it.
- If the control plane fails, the cluster cannot make changes.

2. Scheduler and Controllers

- Scheduler decides which node a Pod runs on.
- Controllers (e.g. Deployment controller) ensure the desired state is maintained.
- They automatically create, replace, or reschedule Pods when needed.

3. API Server and etcd

- API server is the front end for all requests (kubectl, UI, CI/CD, etc).
- etcd is the distributed database that stores the cluster state.
- If etcd is unhealthy, the cluster can behave unpredictably.

4. Core Workloads

- Pods – smallest deployable unit.
- Deployments – manage stateless applications.
- StatefulSets – manage stateful applications.
- DaemonSets – run a Pod on every node.
- Jobs – run tasks to completion.
- CronJobs – run scheduled jobs.

5. Services and Ingress

- Services provide stable network access to Pods.
- Types: ClusterIP, NodePort, LoadBalancer, Headless.
- Ingress manages HTTP/HTTPS routing to services.
- Used with ingress controllers (e.g. Nginx, AWS ALB, Traefik).

6. Requests and Limits

- Requests tell Kubernetes how much resource a Pod needs.
- Limits set the maximum resource a Pod can use.
- Helps with scheduling, performance, and stability.

Example:

```
resources:
  requests:
    cpu: "250m"
    memory: "512Mi"
  limits:
    cpu: "500m"
    memory: "1Gi"
```

7. Scheduling

- Scheduler uses resource requests, node labels, taints/tolerations, and affinity rules.
- You can control where Pods run using:
 - nodeSelector
 - nodeAffinity
 - taints and tolerations
 - topology spread constraints

8. Probes

- Probes check if a container is healthy.
- Types:
 - livenessProbe – restart if unhealthy.
 - readinessProbe – remove from service if not ready.
 - startupProbe – give time to start before checking.

9. Persistent Storage

- Use PersistentVolumes (PV) and PersistentVolumeClaims (PVC).
- Supports different storage classes (e.g. EBS, EFS, Ceph).
- Essential for stateful applications (databases, etc).

10. RBAC (Role-Based Access Control)

- Controls who can do what in the cluster.
- Uses Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings.
- Follow the principle of least privilege.

11. Network Policies

- Control traffic between Pods, namespaces, and external sources.
- Help with security and isolation.
- Commonly used with CNI plugins (e.g. Calico, Cilium).

12. Cluster Upgrades

- Plan upgrades carefully.
- Check version compatibility for Kubernetes, container runtimes, and add-ons.
- Use a rolling upgrade approach.
- Test in a non-production environment first.
- Always have a rollback plan.

13. Node Failures

- Kubernetes detects unhealthy nodes using node conditions.
- Pods are automatically rescheduled to healthy nodes.
- Understand how to handle node drains, cordon, and uncordon.
- Monitor node health and resource usage.

14. Common Failures and How to Debug

Issue	What It Means / How to Debug
CrashLoopBackOff	Container keeps failing. Check logs, config, dependency, resources. Use: <code>kubectl logs <pod></code>
Pending	Pod cannot be scheduled. Check resources, node selectors, taints, PVCs. Use: <code>kubectl describe pod</code>
OOMKilled	Container ran out of memory. Check limits and logs. Use: <code>kubectl logs <pod> --previous</code>
Networking failure	Cannot reach service or external resource. Check services, DNS, network policies, and CNI. Use: <code>kubectl get svc</code> , <code>kubectl get netpol</code>

Useful kubectl Commands

```
✓ kubectl get pods
  kubectl describe pod <pod>
  kubectl logs <pod>
  kubectl get nodes
  kubectl get svc
  kubectl get ingress
  kubectl get pvc
  kubectl get events
  kubectl top nodes
  kubectl top pods
```



KEY TAKEAWAY

Kubernetes is more than kubectl. Understand how it works, plan for failure, and design resilient, secure, and scalable systems.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

SAME
CONTAINERS
BETTER
ENGINEERS



Containers in Production

Senior DevOps engineers use containers to build, ship, and run applications reliably at scale. They understand how containers work under the hood, so they can troubleshoot, secure, and operate them in production.

1. Images vs Containers

- An image is a read-only template.
- A container is a running instance of an image.
- Many containers can be created from the same image.
- Images are built, containers run.



Image
(template)



Container
(running instance)

2. OCI Images

- OCI (Open Container Initiative) is the standard for container images.
- Defines image format and runtime spec.
- Ensures portability across different runtimes (Docker, containerd, CRI-O, etc.).

OCI components:

- Image specification
- Runtime specification
- Distribution specification

3. Image Layers

- Images are made of multiple layers.
- Each layer represents a set of changes.
- Layers are cached to make builds faster.
- Only changed layers are rebuilt.



Application Layer
Dependencies Layer
Runtime Layer
Base OS Layer

4. Container Runtime

- A runtime creates and runs containers from OCI images.
- Examples: containerd (used by Kubernetes), Docker (uses containerd), CRI-O.
- Manages container processes, namespaces, cgroups, and filesystems.

Check runtime on a system:

`docker info`
`containerd --version`

5. Namespaces and cgroups

- Namespaces isolate container processes, network, filesystem, users, etc.
- cgroups (control groups) limit and manage resource usage (CPU, memory, etc.).
- They make containers lightweight and isolated.

View cgroup info:

`cat /sys/fs/cgroup/`

6. Container Networking

- Each container gets its own network namespace.
- Can communicate with other containers, the host, or external networks.
- Uses virtual interfaces and bridges (e.g. docker0).
- Supports different network drivers (bridge, host, overlay, macvlan).

View container network:

`docker network ls`
`docker inspect <container>`

7. Volumes and Persistent Data

- Containers are ephemeral (they can be deleted).
- Volumes are used to persist data outside the container.
- Types: bind mounts, named volumes, and volume drivers.
- Essential for databases and stateful applications.

Example (named volume):

`docker volume create data`
`docker run -v data:/app/data nginx`

8. Registries

- Container images are stored in registries.
- Examples: Docker Hub, AWS ECR, Azure ACR, Google Artifact Registry, JFrog Artifactory.
- Use private registries for production.
- Control access and scan images for vulnerabilities.

Example (pull image):

`docker pull nginx:latest`

9. Image Security

- Scan images for vulnerabilities.
- Use minimal base images (e.g. distroless, alpine).
- Keep images updated.
- Avoid hardcoded secrets.
- Use signed images and trust policies.
- Implement image scanning in CI/CD pipelines.

Scan example:

`trivy image nginx:latest`

10. Rootless Containers

- Run containers without root privileges.
- Reduces security risk if container is compromised.
- Supported by Docker, containerd, and Kubernetes.
- Useful for multi-tenant environments.

Run rootless container:

`docker run --user 1000:1000 nginx`

11. Resource Limits

- Set CPU and memory limits to prevent resource exhaustion.
- Use requests and limits (in Kubernetes).
- Helps with stability and fair resource allocation.

Example (Docker):

`docker run -d \`
`--cpus=1 \`
`--memory=512m`
`nginx`

12. Container Lifecycle

- Created → Started → Running → Stopped → Removed.
- Important commands: run, start, stop, restart, rm, logs, exec, inspect.
- Understand container states and exit codes.

Common commands:

`docker ps -a`
`docker logs <container>`
`docker exec -it <container> sh`

13. Why Containers Unexpectedly Exit

- Application crashes (check logs).
- Out of memory (OOMKilled).
- Health check failures.
- Incorrect configuration or environment variables.
- Dependency failures (database, network, etc.).
- Process exits after completing its task.

14. Debugging Containers (The Right Way)

- Check container status and logs.
- Use docker logs, docker inspect, and docker exec.
- Check resource usage (CPU, memory).
- Inspect the application and environment.
- Verify networking and dependencies.
- Avoid treating containers like full VMs.
- Use minimal, focused debugging steps.

Senior Engineer Mindset

- ✓ Understand the underlying technology.
- ✓ Keep images small, secure, and updated.
- ✓ Design for observability and easy debugging.
- ✓ Use least privilege (rootless where possible).
- ✓ Containers are ephemeral, so externalize state.
- ✓ Think about security, performance, and cost.



KEY TAKEAWAY

Containers are powerful because they are lightweight, portable, and scalable. Understanding how they work helps you run them securely and reliably in production.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA

ENGINEERS TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW



Code → Build → Test
→ Deploy → Operate

CI/CD as a Production System

Senior DevOps engineers treat CI/CD as a production system. They design, secure, monitor, and continuously improve their pipelines to deliver reliable software at scale.

**AUTOMATE
BUILD
TEST
DEPLOY
DELIVER
REPEAT**

1. Pipeline Architecture

- A CI/CD pipeline automates the entire software delivery process.
- Consists of stages and jobs.
- Should be reliable, secure, scalable, and observable.



Code Build Test Package Deploy

2. Build, Test, Scan, Package, Deploy, Verify

- Typical pipeline stages:

- 1 Build – compile code
- 2 Test – run unit and integration tests
- 3 Scan – security and quality checks
- 4 Package – create immutable artifact
- 5 Deploy – deploy to target environment
- 6 Verify – confirm the deployment is working

3. Artifact Immutability

- Build artifacts should not change after they are created.
- Use versioned, immutable artifacts (e.g. JAR, Docker image).
- Store artifacts in a registry (e.g. JFrog Artifactory, ECR, ACR, GCR).
- Use the same artifact across environments (dev → staging → prod).



Build Once
Deploy Everywhere

4. Dependency Management

- Use dependency management tools (Maven, npm, etc).
- Pin versions to avoid unexpected changes.
- Regularly update dependencies.
- Scan for vulnerable dependencies.
- Use artifact repositories (e.g. JFrog Artifactory, Nexus).

Example (Maven):

```
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-core</artifactId>
<version>6.1.0</version>
</dependency>
```

5. Secrets and Credentials

- Never hardcode secrets in code or pipelines.
- Use secret managers (e.g. HashiCorp Vault, AWS Secrets Manager, Azure Key Vault).
- Use environment variables or secure injection.
- Rotate credentials regularly.
- Limit access with least privilege.



Keep Secrets Secret

6. Runners and Agents

- Runners/agents execute pipeline jobs.
- Can be self-hosted or cloud-hosted (e.g. GitHub Actions, GitLab CI, Jenkins).
- Manage capacity, security, and isolation.
- Use ephemeral runners for better security.
- Monitor runner health and usage.



Runners Execute
Your Pipelines

7. Environment Promotion

- Deploy the same artifact across environments.
- Use a clear promotion flow: Dev → Staging → Production.
- Keep environment configurations separate (not mixed in code).
- Use infrastructure as code (Terraform, etc).
- Apply automated and manual checks before promotion.



8. Deployment Approvals

- Use manual approvals for production.
- Define who can approve.
- Use policy-based approvals (e.g. security, compliance).
- Automate checks (tests, scans) before approval.
- Keep an audit trail of approvals.



Right People
Right Changes

9. Pipeline Concurrency

- Control how many pipeline runs can execute at the same time.
- Avoid resource contention (e.g. shared environments).
- Use concurrency limits, queues, or environments.
- Cancel redundant runs (e.g. older commits).



Control Concurrency
For Stable Pipelines

10. Failed Deployments

- Understand why the deployment failed (logs, events, health checks).
- Automate rollback if needed.
- Notify the right people.
- Keep the system in a safe state.
- Learn from failures and improve the pipeline.



Failures Happen
Be Ready

11. Rollbacks

- Have a clear rollback strategy.
- Use previous immutable artifacts.
- Automate rollback where possible.
- Verify the rollback was successful.
- Test rollback procedures regularly.



Always Have
A Rollback Plan

12. Pipeline Observability

- Monitor pipeline success and failure rates.
- Track build and deployment times.
- Alert on failures.
- Monitor runner/agent health.
- Use dashboards (e.g. Grafana, Datadog).



Observe
Your Pipelines

13. Securing the Software Supply Chain

- Scan code, dependencies, and container images.
- Use signed commits and artifacts.
- Verify provenance (e.g. SLSA, in-toto).
- Use trusted base images.
- Implement least privilege in pipelines.
- Prevent tampering and ensure integrity from code to production.



KEY TAKEAWAY

A CI/CD pipeline is not just a tool.
It is a critical production system.
Design it to be reliable, secure,
observable, and scalable.

SENIOR ENGINEER MINDSET

- ✓ Automate and simplify.
- ✓ Think about security and compliance.
- ✓ Design for failure and recovery.
- ✓ Monitor and improve continuously.
- ✓ Treat your pipeline like a product.
- ✓ Deliver value, not just deployments.



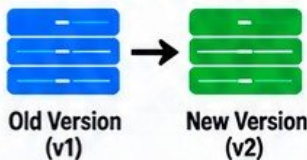
Production Deployment Strategies

PLAN
DEPLOY
VERIFY
ROLLBACK
STAY IN
CONTROL

Senior DevOps engineers choose the right deployment strategy for the right situation. They focus on reliability, minimize risk, and ensure business continuity during every deployment.

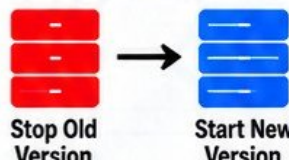
1. Rolling Deployments

- Gradually replace old instances with new ones.
- No downtime.
- Commonly used for stateless applications.
- Works well with load balancers and orchestration tools (e.g. Kubernetes).
- Monitor health during rollout.



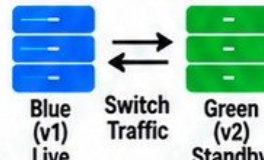
2. Recreate Deployments

- Terminate all existing instances and start new ones.
- Simple but causes downtime.
- Usually used for non-critical environments or when state cannot be shared.
- Ensure proper backup and drain connections before recreating.



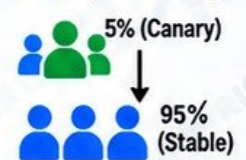
3. Blue-Green Deployments

- Maintain two identical environments (Blue and Green).
- Deploy new version to the inactive environment.
- Switch traffic after verification.
- Quick rollback by switching back.
- Requires more infrastructure but reduces risk.



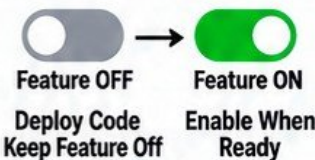
4. Canary Deployments

- Release new version to a small percentage of users.
- Monitor metrics, logs, and errors.
- Gradually increase traffic if healthy.
- Reduce or stop if issues are detected.
- Ideal for high-risk changes.



5. Feature Flags

- Enable or disable features without redeploying code.
- Roll out features to specific users or environments.
- Helps with testing and gradual release.
- Quickly disable a feature if issues occur.



6. Progressive Delivery

- Use a combination of strategies (canary, feature flags, A/B testing).
- Gradually roll out changes based on real user feedback.
- Monitor business and technical metrics.
- Minimizes risk and improves confidence.



7. Database Compatibility During Deployment

- Design database changes to be backward compatible.
- Use expand and contract pattern (add new, then remove old).
- Avoid breaking changes.
- Coordinate application and database deployments.
- Test migrations in staging first.



8. Backward Compatibility

- Ensure new version works with older clients and data.
- Use versioning (APIs, schemas).
- Avoid breaking changes to contracts.
- Maintain support for previous versions during transition.
- Essential for microservices and distributed systems.



9. Health Verification

- Check application health after deployment.
- Monitor metrics, logs, and traces.
- Verify key user flows (synthetic and real).
- Use automated and manual checks.
- Do not rely only on "deployment success".



Verify
Before Full Traffic

10. Automated Rollback

- Set clear failure conditions (e.g. error rate, latency, health checks).
- Automatically roll back to previous version.
- Use monitoring and alerting to trigger rollback.
- Test rollback process regularly.
- Keep previous version available and immutable.



Detect Issues
Roll Back Fast

11. Deployment Blast Radius

- Understand what will be affected by the deployment.
- Limit the blast radius using strategies like canary or regional rollout.
- Consider dependencies (databases, queues, APIs).
- Have a rollback plan.
- Start with smaller scope when possible.



Smaller Changes
Lower Risk

12. Choosing Strategies Based on Risk, Not Fashion

- There is no one-size-fits-all strategy.
- Consider the business impact, user base, and system complexity.
- Choose the simplest strategy that meets the risk level.
- Do not use blue-green or canary just because it is popular.
- Focus on reliability, safety, and business needs.



Choose What Fits
Your Risk Level



KEY TAKEAWAY

A successful deployment is not just about pushing code. It is about delivering value safely, minimizing risk, and keeping the business running.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA

ENGINEERS TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW


VERIQTA

OBSERVE TODAY
A BETTER TOMORROW

Observability and Production Signals

Senior DevOps engineers use observability to understand what is happening in production, find problems faster, and keep systems reliable, secure, and performant.

SEE
UNDERSTAND
TROUBLESHOOT
SOLVE
IMPROVE
REPEAT

1. Metrics

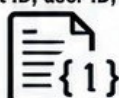
- Numerical data collected over time.
- Shows system health, performance, and resource usage.
- Examples: CPU, memory, request count, latency, errors.
- Use time series data (e.g. Prometheus).



Metrics = Trends

2. Logs

- Time-stamped text records of events.
- Help you understand what happened.
- Use structured logging (e.g. JSON).
- Include useful context (request ID, user ID, service).



Logs = Detail

3. Traces

- Track a request as it travels across multiple services.
- Show the full path and latency of each step.
- Help identify where time is spent and where failures occur.
- Use tools like OpenTelemetry, Jaeger, or Tempo.



Traces = End-to-End View

4. Events

- Discrete signals that something happened (informational, warning, error).
- Examples: pod restarted, deployment failed, instance terminated.
- Help with incident investigation and audit trails.
- Often found in Kubernetes events, cloud provider events, and system logs.



Events = State Changes

5. Correlation

- Link metrics, logs, traces, and events together.
- Use a common identifier (e.g. trace ID, request ID).
- Helps you see the full story across services.



Correlation = Complete Picture

6. RED Method

- Focus on the user perspective (for services).
- R** = Request rate
- E** = Error rate
- D** = Duration (latency)
- Use for SLOs and alerting on services.

R	Request Rate
E	Error Rate
D	Duration (Latency)

RED = Service Health

7. USE Method

- Focus on resource utilization (for infrastructure).
- U** = Utilization
- S** = Saturation
- E** = Errors
- Use for CPU, memory, disk, network, etc.

U	Utilization
S	Saturation
E	Errors

USE = Resource Health

8. Golden Signals

- A Google-recommended set for monitoring services.
- Latency (how fast)
- Traffic (how much)
- Errors (failure rate)
- Saturation (resource limits)



Golden Signals = Reliable Services

9. Structured Logging

- Use a consistent format (e.g. JSON).
- Include timestamps, log level, service name, and context.
- Makes logs easy to search, filter, and analyze.

```
{
  "timestamp": "2026-01-01T12:00:00Z",
  "level": "INFO",
  "service": "api",
  "message": "User login successful"
}
```

Structured = Searchable

10. Distributed Tracing

- Track requests across multiple services.
- Visualize the full request flow.
- Identify latency and failures in complex systems.
- Use OpenTelemetry, Jaeger, or cloud-native tools.



Tracing = Follow the Request

11. OpenTelemetry

- Open standard for collecting metrics, logs, and traces.
- Vendor neutral.
- Works with many tools (Prometheus, Grafana, Jaeger, Datadog, etc.).
- Helps avoid vendor lock-in.



Standard = Flexibility

12. Dashboards

- Visualize key metrics, logs, traces, and system health.
- Use tools like Grafana, Kibana, or cloud dashboards.
- Keep dashboards simple and focused.
- Show real-time and historical data.



Dashboards = Visibility

13. Alerting

- Notify the right people when something is wrong.
- Use meaningful thresholds and reduce noise.
- Alert on symptoms and user impact, not just infrastructure.
- Integrate with Slack, PagerDuty, etc.



Alerting = Faster Response

14. Cardinality

- Cardinality is the number of unique label values.
- High cardinality can increase costs and degrade performance.
- Avoid using user IDs, session IDs, or other high-cardinality labels.
- Use low-cardinality, meaningful labels.



Control Cardinality = Lower Cost

15. Observability Cost

- Metrics, logs, and traces can be expensive at scale.
- Control data volume, retention, and cardinality.
- Use sampling for traces.
- Choose the right storage tier.
- Regularly review and clean up unused data.



Monitor Costs = Sustainable

16. Moving from Symptom to Root Cause

- Start with the symptom (e.g. high latency, errors).
- Use metrics, logs, traces, and events to investigate.
- Correlate data across services.
- Form a hypothesis, test it, and find the root cause.
- Document the fix and prevent recurrence.



From Symptom to Root Cause = Real Impact



KEY TAKEAWAY

Observability is not just about collecting data. It is about turning data into insight, finding problems faster, and building more reliable systems.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQT

ENGINEERS TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW


RELIABLE SYSTEMS
HAPPIER PEOPLE

SRE and Reliability Engineering

RELIABILITY
SCALES
TRUST
BUILDS
OPPORTUNITY

Senior DevOps engineers use SRE principles to build and operate reliable systems at scale. Reliability is not an accident, it is a design choice.

1. SLIs (Service Level Indicators)

- SLIs are measurable indicators of service performance.
- Examples: availability, latency, throughput, error rate.
- Choose SLIs that reflect customer experience.



2. SLOs (Service Level Objectives)

- SLOs are target values for SLIs over a time period.
- They define the level of service you aim to provide.
- Example: 99.9% availability in a month.



3. SLAs (Service Level Agreements)

- SLAs are formal contracts (often with customers).
- They define the consequences if SLOs are not met.
- SLAs usually include uptime guarantees and financial penalties.



4. Error Budgets

- Error budget is the amount of failure you can afford while still meeting your SLO.
- Helps balance reliability and feature delivery.
- When the budget is used up, focus shifts to reliability, not new features.



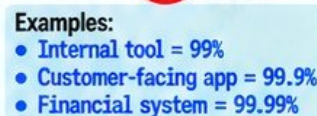
5. Availability Calculations

- Availability = (Total time - Downtime) / Total time × 100%.
- Small differences matter at scale.
- Examples (monthly):
- 99% = 7.2 hours downtime
- 99.9% = 43 minutes downtime
- 99.99% = 4.3 minutes downtime
- 99.999% = 26 seconds downtime



6. Reliability Targets

- Choose realistic targets based on business needs.
- Higher reliability costs more (time, money, complexity).
- Different services may have different targets.



7. Toil

- Toil is manual, repetitive work that adds no long-term value.
- Examples: manual restarts, manual deployments, responding to repetitive alerts.
- SREs reduce toil through automation, better tooling, and system improvements.



8. Capacity Planning

- Ensure systems can handle current and future demand.
- Consider CPU, memory, storage, network, and traffic growth.
- Use metrics and trends to predict needs.



9. Reliability vs Delivery Speed

- Faster delivery can increase risk if not managed well.
- SREs find the right balance between shipping features and maintaining reliability.
- Use error budgets and risk assessment to make decisions.



10. Redundancy

- Use multiple instances, zones, or regions.
- Eliminate single points of failure.
- Examples: multi-AZ, multi-region, load balancers, replication.



11. Graceful Degradation

- When something fails, the system continues to work with reduced functionality instead of a complete outage.
- Examples: show cached data, disable non-critical features.



12. Reliability as an Engineering Constraint

- Reliability must be considered in design, architecture, and development.
- Make tradeoffs explicitly.
- Include reliability in code reviews, design docs, and planning.



13. Why 100% Availability Is Usually the Wrong Target

- 100% availability is extremely expensive and unrealistic.
- All systems can and will fail (hardware, software, humans, etc.).
- Chasing 100% can slow down innovation and waste resources.
- A more realistic target allows for sustainable growth.
- Focus on reliability, resilience, and fast recovery instead.

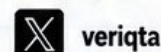


KEY TAKEAWAY

Reliability is a journey, not a number. Use SLOs, error budgets, automation, and good engineering practices to build systems that are reliable, scalable, and sustainable.

SRE MINDSET

- Think long term.
- Automate and eliminate toil.
- Design for failure.
- Measure what matters.
- Balance reliability and velocity.
- Keep the customer experience in focus.
- Continuous improvement.



VERIQT A

ENGINEERS TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW



INCIDENTS HAPPEN
PREPARED ENGINEERS
HANDLE THEM BETTER

Incident Response and On-Call

PREPARE
RESPOND
RECOVER
LEARN
IMPROVE

Senior DevOps engineers respond to incidents quickly, calmly, and systematically. Good incident response reduces impact, restores service, and prevents recurrence.

1. Detection

- Detect incidents through monitoring, alerts, logs, user reports, or third-party notifications.
- Use well-tuned alerts to avoid noise.
- Confirm that it is a real issue and not a false alarm.
- Capture initial signals (metrics, logs, traces, events).



Detect Early
Reduce Impact

2. Triage

- Quickly understand the issue.
- Check what is affected, when it started, and the symptoms.
- Gather initial information from monitoring, logs, and user reports.
- Determine if it is an incident or a normal issue.



Understand Fast
Take the Right Actions

3. Severity Classification

- Classify severity based on business impact.
- Common levels:
 - P1** Critical (major outage)
 - P2** High (significant impact)
 - P3** Medium (limited impact)
 - P4** Low (minimal impact)
- Use clear criteria to ensure consistent classification.



Severity Drives Response

4. Incident Command

- Assign an incident commander (IC) to lead the response.
- Define roles and responsibilities (operations, communication, scribes, etc.).
- Keep decision-making clear and centralized.
- Use established runbooks when available.



Clear Leadership
Reduces Chaos

5. Communication

- Communicate early, clearly, and regularly.
- Keep stakeholders, users, and teams informed.
- Be honest about what you know and what you don't know.
- Use predefined communication channels and templates.



Clear and Timely
Communication Builds Trust

6. Mitigation Before Root Cause

- Focus on reducing user impact first.
- Use workarounds, scaling, failover, or feature disablement as needed.
- You do not need the root cause to start mitigating.



Reduce Impact Now
Find Root Cause Next

7. Escalation

- Escalate when the incident is beyond your scope.
- Follow the escalation matrix (on-call, senior engineers, vendors, etc.).
- Do not hesitate to call for help early.
- Keep stakeholders informed when escalating.



Escalate Early
Get the Right Support

8. Rollback Decisions

- Know when to roll back to a previous version.
- Consider user impact, data risk, and system state.
- Have a rollback plan ready before deployments.
- Validate the rollback and monitor after execution.



When in Doubt
Roll Back Safely

9. Evidence Collection

- Collect logs, metrics, traces, configuration changes, and relevant data.
- Preserve evidence for analysis.
- Include timestamps and screenshots.
- Store evidence in a central location.



Collect Facts
Not Assumptions

10. Timeline Creation

- Record a detailed timeline of events.
- Include when the issue started, who did what, and when.
- Use data from logs, alerts, chat, and systems.
- A clear timeline helps with root cause analysis and postmortems.



A Good Timeline
Tells the Real Story

11. Recovery

- Restore full service.
- Verify that the system is stable and working as expected.
- Monitor closely for any recurrence.
- Communicate that the incident is resolved.



Recover Completely
Before Closing

12. Post-Incident Review

- Conduct a blameless postmortem.
- Identify what happened, why it happened, and what can be improved.
- Focus on systems, processes, and prevention (not individuals).
- Create action items with owners and timelines.



Learn, Improve
and Prevent Recurrence

13. Blameless Postmortems

- Focus on learning, not blaming.
- Encourage open and honest discussion.
- Identify contributing factors.
- Turn lessons into actionable improvements.
- Share learnings across the organization.



Blame Less
Learn More

14. Preventing Recurrence

- Implement fixes and improvements.
- Improve monitoring, alerting, and testing.
- Address root causes, not just symptoms.
- Update runbooks and documentation.
- Track action items to completion.



Fix the Cause
Not Just the Issue

15. Staying Disciplined During a Major Outage

- Stay calm and focused.
- Follow the incident process.
- Avoid making changes without understanding the impact.
- Communicate frequently.
- Take breaks if needed.
- Support your team and keep a positive mindset.

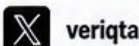


Discipline
Leads to a Better Outcome



KEY TAKEAWAY

Incidents are inevitable. A structured response, clear communication, and continuous learning turn incidents into stronger, more reliable systems.



VERIQT A

ENGINEERS TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW



VERIQTA

FIND PROBLEMS
FIX PROBLEMS
MAKE SYSTEMS BETTER

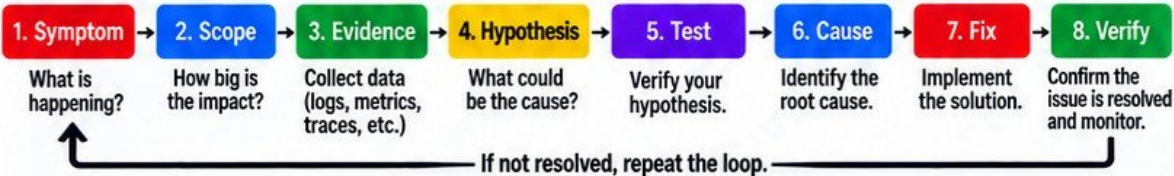
Production Troubleshooting

Senior DevOps engineers use a structured and methodical approach to troubleshoot production issues. Be calm, think clearly, gather evidence, and fix the right problem.

OBSERVE
ANALYZE
TROUBLESHOOT
RESOLVE
LEARN
IMPROVE

1. The Senior Troubleshooting Loop

Follow a structured process instead of guessing. This helps you find the real root cause faster.



“Good engineers find the cause. Great engineers prevent it from happening again.”

— VERIQTA

2. What Changed?

- Check recent deployments.
- Look for configuration changes.
- Review infrastructure changes (e.g. scaling, instance replacement).
- Check recent code commits.
- Identify changes in dependencies (e.g. database, third-party services).
- Use change logs, deployment tools, and monitoring timelines.



Recent Changes Often Explain the Problem

3. Determine Blast Radius

- Is it affecting one user, one service, or the entire system?
- Which regions, AZs, or clusters are impacted?
- Is it limited to a specific feature or global?
- Identify affected users, services, and dependencies.



Understand the Impact Before You Act

4. Application vs Infrastructure Failures

- Application failures: bugs, memory leaks, unhandled exceptions, bad requests.
- Infrastructure failures: compute, network, storage, load balancers, DNS, cloud provider issues.
- Check both layers. It can be one or a combination.



Application
(Code, Logic, Data)



Infrastructure
(Servers, Network, Cloud)

5. Logs, Metrics, Traces and Events

- Logs: look for errors and patterns.
- Metrics: check CPU, memory, latency, error rates, etc.
- Traces: follow the request across services.
- Events: check system and cloud events (e.g. AWS Health, Kubernetes events).
- Correlate all signals using timestamps.



Logs Metrics Traces Events

6. Dependency Failures

- Check external services (databases, APIs, message queues, caches).
- Verify service endpoints, DNS, and connectivity.
- Look for authentication, rate limits, or timeouts.
- Check if a dependency is slow or returning errors.



A Downstream Service Can Break Your System

7. Network Failures

- Check DNS resolution.
- Verify connectivity and routing (e.g. VPC, subnets, security groups).
- Look for firewall or network policy blocking traffic.
- Check load balancers and health checks.
- Use tools like ping, traceroute, curl, and VPC flow logs.



No Network
No Service

8. Resource Exhaustion

- Check CPU, memory, disk, and network utilization.
- Look for memory leaks, high GC, or disk full.
- Verify pod/instance limits and quotas.
- Check for resource contention (e.g. noisy neighbor).



When Resources Run Out
Systems Fail

9. Configuration Changes

- Review application and infrastructure configuration.
- Check environment variables, feature flags, and secrets.
- Look for misconfigurations in IDs, endpoints, or permissions.
- Validate configuration management tools (e.g. Terraform, Ansible).



A Small Misconfiguration Can Cause a Big Outage

10. Deployment-Related Failures

- Check deployment status and rollout history.
- Verify if new version introduced a bug.
- Look for failed or partial deployments.
- Confirm database schema changes.
- Check if rollback is possible and safe.



Deployments Can Introduce Risk
Always Monitor After a Release

11. Avoid Changing Five Things Simultaneously

- Change one thing at a time.
- Otherwise you won't know what fixed it.
- Keep a record of every change.
- Use a rollback plan.
- Be methodical and patient.



One Change at a Time
Find the Real Fix

12. Preserve Evidence Before Restarting Everything

- Collect logs, metrics, and traces first.
- Take screenshots and save error messages.
- Capture system state (e.g. describe commands).
- Store evidence in a central location.
- Do not immediately restart services unless necessary.



Evidence Helps You Find the Real Cause

13. Key Takeaways

- Use a structured approach.
- Understand what changed.
- Determine the blast radius.
- Check application and infrastructure.
- Use logs, metrics, traces, and events.
- Investigate dependencies, network, resources, and configurations.
- Avoid changing multiple things.
- Preserve evidence.
- Learn and prevent recurrence.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

VERIQTA

ENGINEERS TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW



VERIQTA
SECURE TODAY
SCALE TOMORROW

Security Is Part of DevOps

Security is not a separate team's job. It is everyone's responsibility.
Build, deploy, and operate securely from day one.

SECURE
AUTOMATE
MONITOR
COMPLY
SCALE
TOGETHER

1. Least Privilege

- Give only the minimum access needed to do the job.
- Reduce permissions, scope, and duration.
- Review access regularly.
- Remove unused permissions.



Minimum Access
Maximum Security

2. IAM (Identity and Access Management)

- Manage users, roles, and policies.
- Use roles instead of long-lived credentials.
- Apply MFA for human users.
- Use fine-grained permissions and conditions.



Control Access
Protect Resources

3. Workload Identity

- Use service accounts and IAM roles for workloads (e.g. Kubernetes, AWS IAM Roles for Service Accounts).
- Avoid hardcoded credentials in code or images.
- Use short-lived, automatically rotated credentials.



Give Identities to Workloads
Not Passwords in Code

4. Secrets Management

- Store secrets in a dedicated secrets manager (e.g. AWS Secrets Manager, HashiCorp Vault).
- Do not store secrets in code, images, or configuration files.
- Use environment variables or secure mounts.
- Restrict access and audit usage.



Keep Secrets Secret

5. Credential Rotation

- Rotate access keys, tokens, and passwords regularly.
- Use automated rotation where possible.
- Revoke unused or compromised credentials immediately.
- Monitor and alert on unusual credential usage.



Rotate Regularly
Reduce Risk

6. Encryption at Rest and in Transit

- Encrypt data at rest (e.g. EBS, S3, RDS, databases).
- Encrypt data in transit using TLS.
- Use managed encryption services and proper key management.
- Do not disable encryption for convenience.



Encrypt Everything
Everywhere

7. TLS (Transport Layer Security)

- Use TLS for all services and APIs.
- Enforce modern TLS versions (e.g. 1.2 or 1.3).
- Manage certificates properly (e.g. automated renewal).
- Avoid self-signed certificates in production.



Keep Connections Secure

8. Network Segmentation

- Separate environments (e.g. dev, staging, production).
- Use VPCs, subnets, and security groups.
- Apply the principle of least access between tiers.
- Restrict access to only what is needed.



Isolate and Protect

9. Container Security

- Use minimal base images.
- Scan images for vulnerabilities.
- Avoid running containers as root.
- Use read-only file systems where possible.
- Apply security policies (e.g. seccomp, AppArmor).



Build Secure Containers

10. Kubernetes Security

- Use RBAC and least privilege for service accounts.
- Enable pod security standards (e.g. restricted).
- Use network policies.
- Keep the cluster and components up to date.
- Scan images and monitor for threats.



Secure the Cluster
Secure the Workloads

11. Dependency and Image Scanning

- Scan dependencies (e.g. Snyk, Trivy, SonarQube).
- Scan container images before deployment.
- Check for known vulnerabilities (CVEs).
- Fail the build if critical vulnerabilities are found.



Find and Fix Vulnerabilities Early

12. Infrastructure Security

- Secure cloud infrastructure (e.g. VPC, storage, compute).
- Use Infrastructure as Code (IaC) with security best practices.
- Scan IaC templates for misconfigurations.
- Apply patching and regular updates.



Secure by Design
From Infrastructure to Application

13. Audit Logging

- Enable logs for all critical services (e.g. API, infra, access).
- Send logs to a central system (e.g. CloudWatch, ELK, SIEM).
- Monitor for suspicious activity.
- Keep logs immutable and retained for compliance.



Logs Tell the Truth

14. Supply-Chain Security

- Use trusted sources for images and packages.
- Verify image signatures (e.g. Cosign, Notary).
- Pin versions and check checksum.
- Scan third-party components.
- Monitor for compromised dependencies.



Trust but Verify

15. Never Trade Security for a Quick Production Fix

- Temporary shortcuts create long-term risk.
- Understand the risk before any change.
- Document and track exceptions.
- Fix the root cause properly.
- Security and speed can work together with the right process.



Shortcuts Today
Bigger Incidents Tomorrow

KEY TAKEAWAYS

- Security is everyone's job.
- Use least privilege.
- Protect identities and secrets.
- Encrypt data.
- Scan and monitor continuously.
- Follow secure practices in code, containers, cloud, and Kubernetes.
- Keep good logs and audit trails.
- Never ignore security for speed.
- A secure system is a reliable system.
- Build security into your DevOps culture.



VERIQTA
DATA POWERS
APPLICATIONS
RELIABILITY KEEPS
THEM RUNNING

Databases, Caches, and Stateful Systems

Senior DevOps engineers understand data, because state changes everything.
Reliable systems require reliable data.

DATA
PERSISTS
SYSTEMS
SCALE
BUSINESSES
GROW

1. Relational vs NoSQL

- Relational (e.g. PostgreSQL, MySQL): structured data, ACID, SQL, strong consistency.
- NoSQL (e.g. MongoDB, DynamoDB): flexible schema, horizontal scale, different consistency models.
- Choose based on use case, not trends.



Different Tools for Different Needs

2. Connections and Connection Pools

- Databases have a limited number of connections.
- Use connection pools (e.g. HikariCP, PgBouncer).
- Avoid creating a new connection per request.
- Monitor and tune pool size.



Pool Connections
Improve Performance

3. Transactions

- Ensure data consistency (ACID: Atomicity, Consistency, Isolation, Durability).
- Use transactions for related changes.
- Be aware of transaction isolation levels.
- Keep transactions short to avoid locks.



Transactions Keep Data Reliable

4. Indexes

- Indexes speed up data retrieval.
- Use the right index for your query patterns.
- Too many indexes can slow down write operations.
- Monitor and remove unused indexes.



Right Indexes
Faster Queries

5. Replication

- Keep multiple copies of data across nodes or regions.
- Used for high availability and read scaling.
- Can be synchronous or asynchronous.



Replicate for Availability

6. Read Replicas

- Offload read traffic to replicas.
- Improve application performance.
- Be aware of replication lag (eventual consistency).



Scale Reads, Protect Primary

7. Failover

- Automatically switch to a healthy node when the primary fails.
- Use managed solutions or tools (e.g. Patroni, RDS Multi-AZ).
- Test failover regularly.



Be Ready for Failures

8. Backups and Restores

- Take regular automated backups.
- Test your restore process.
- Store backups in a different region or account.
- Know your Recovery Point Objective (RPO) and Recovery Time Objective (RTO).



Backups Save Businesses

9. Database Migrations

- Plan migrations carefully.
- Use zero-downtime techniques (e.g. blue-green, rolling).
- Test in a staging environment first.
- Be ready to rollback if needed.



Migrate Safely
Keep Services Running

10. Redis and Caching

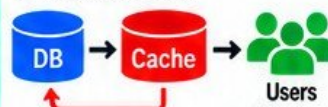
- Redis is an in-memory data store used for caching, sessions, queues, etc.
- Reduces load on your database.
- Use the right data structures (strings, hashes, lists, etc.).
- Monitor memory usage and evictions.



Cache Smart
Serve Faster

11. Cache Invalidation

- Keep cache and database in sync.
- Use strategies like TTL (time-to-live), event-driven invalidation, or write-through.
- Avoid stale data.



Fresh Data, Happy Users

12. Data Persistence

- Understand how data is stored and persisted (disks, volumes, snapshots).
- Choose the right storage type (e.g. EBS, EFS, SSD, NVMe).
- Consider durability, availability, and performance.



Persistent Data
Survives Failures

13. Storage Latency

- Storage latency affects database and application performance.
- Use the right storage class (e.g. SSD vs HDD).
- Monitor I/O latency and IOPS.
- Consider placement, network, latency, and disk throughput.



Lower Latency
Better Performance

14. Database Bottlenecks

- Common bottlenecks: slow queries, missing indexes, connection limits, locks, high I/O.
- Use monitoring tools (e.g. Performance Insights, pg_stat, --slow query logs).
- Identify and fix the real cause, not just the symptom.



Find the Bottleneck
Fix the Real Issue

15. Why State Changes Infrastructure Decisions

- Stateful systems need persistent storage, backup, and replication.
- They affect scaling, deployment strategies, and disaster recovery.
- State increases complexity and operational risk.
- Always consider data requirements when designing infrastructure.



State Drives Architecture



KEY TAKEAWAYS

- Choose the right database for your use case.
- Plan for high availability and backups.
- Monitor performance (queries, I/O, latency).
- Use caching to improve speed and reduce load.
- Test failover and restore processes.
- State changes everything. Design infrastructure with data in mind.
- Reliable data means reliable systems.



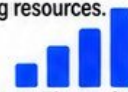
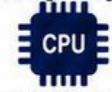
Never ignore database and data risks.
A small data issue can become a major outage.



Performance and Capacity Engineering

Good engineers make it work. Great engineers make it fast, reliable, and scalable.
Measure. Plan. Optimize. Grow.

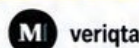
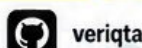
PERFORMANCE
TODAY
RELIABILITY
TOMORROW
SCALE
ALWAYS

1. Latency - Latency is the time it takes for a request to get a response. - Measure end-to-end latency (application, network, database). - High latency affects user experience. - Identify where latency is introduced.  Lower Latency Happier Users	2. Throughput - Throughput is the number of requests or tasks processed in a given time. - Measure requests per second (RPS) or transactions per second (TPS). - Increase throughput by removing bottlenecks and scaling resources.  More Throughput Serve More Users	3. Saturation - Saturation happens when a resource is at or near its limit. - Leads to higher latency, errors, and timeouts. - Monitor CPU, memory, disk, and network utilization. - Add capacity or optimize before saturation impacts users.  High Utilization = Higher Risk	4. CPU - CPU handles application processing and system tasks. - High CPU can cause slow responses and timeouts. - Monitor CPU usage, load average, and throttling. - Look for CPU spikes and sustained high usage.  Keep CPU Healthy Keep Systems Responsive
5. Memory - Memory stores data for fast access. - Memory leaks can cause slowdown or crashes. - Monitor memory usage, swap, and out-of-memory (OOM) events. - Right-size memory and fix leaks.  Enough Memory Stable Performance	6. Disk I/O - Disk I/O affects read and write performance. - High disk latency can slow databases and applications. - Monitor IOPS, latency, and throughput. - Use the right storage type (e.g. SSD, gp3, io2) for your workload.  Fast Disk Faster Systems	7. Network Throughput - Network throughput is the amount of data transferred per second. - Monitor bandwidth, packet loss, and network latency. - Network bottlenecks can impact application performance. - Ensure proper network design, load balancers, and routing.  A Healthy Network Keeps Everything Connected	8. Connection Limits - Every system has a limit on concurrent connections (e.g. database, web server). - Exceeding limits can cause timeouts and errors. - Tune connection pools and adjust limits. - Monitor active and idle connections.  Manage Connections Avoid Outages
9. Queue Depth - Queues hold requests when systems are busy. - Monitor queue length and processing time. - Growing queues can indicate a downstream bottleneck. - Set alerts for abnormal queue growth.  Watch the Queue Spot Problems Early	10. Bottlenecks - Bottlenecks limit overall system performance. - Can be in CPU, memory, disk, network, database, or external services. - Use metrics, tracing, and profiling to find bottlenecks. - Fix the real bottleneck, not just the symptoms.  Find and Remove Bottlenecks Unlock Performance	11. Load Testing - Simulate real traffic to test system behavior. - Identify bottlenecks and breakpoints. - Test for normal load, peak load, and failure scenarios. - Use tools like k6, JMeter, or Locust.  Test Today Handle Tomorrow	12. Capacity Planning - Estimate future resource needs based on growth. - Use historical data and trends. - Consider seasonal spikes and business events. - Plan for headroom to avoid last-minute scaling.  Plan Capacity Stay Ahead of Demand
13. Autoscaling Signals - Use real metrics to trigger scaling (CPU, memory, custom metrics, queue depth). - Avoid over-scaling and under-scaling. - Set appropriate thresholds and cooldown periods. - Test your autoscaling policies.  Scale Automatically When Needed	14. Resource Requests and Limits - Set resource requests to reserve capacity. - Set limits to prevent resource abuse. - Helps with stability, fairness, and cost control (e.g. Kubernetes). - Monitor and adjust based on real usage.  Right Size Resources Run Stable Systems	15. Performance Baselines - Establish baseline metrics when the system is healthy. - Track normal performance patterns (e.g. latency, CPU, RPS). - Use baselines to detect anomalies and regressions. - Update baselines as the system evolves.  Know Your Normal Detect Abnormal Faster	16. Know What Normal Looks Like - You cannot troubleshoot effectively without knowing normal behavior. - Baseline key metrics, logs, and patterns. - Compare current behavior to normal to quickly spot issues. - Always ask: Is this normal for our system?  Know Normal Diagnose Faster

KEY TAKEAWAY

Performance and capacity are not one-time tasks. They are continuous processes. Measure, plan, optimize, and improve.

"You cannot improve what you do not measure."
— VERIQT



VERIQT

ENGINEERS TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
BUILD
GROW



VERIQTA
PREPARE TODAY
BE RESILIENT
STAY ONLINE

Resilience and Disaster Recovery

Systems will fail. Resilient engineers prepare, test, and recover.
Hope is not a strategy. Resilience is a design choice.

PLAN
TEST
RECOVER
ADAPT
STAY
STRONG

1. Failure Domains

- Understand what can fail (e.g. instance, AZ, region, service, network, power).
- Design to limit the blast radius of failures.
- Keep critical components in separate failure domains.



Separate Failure Domains
Reduce Risk

2. Redundancy

- Remove single points of failure.
- Use multiple instances, AZs, or regions.
- Redundancy increases availability and fault tolerance.



No Single Point
of Failure

3. Backups

- Back up data, configuration, and critical resources.
- Follow the 3-2-1 rule (3 copies, 2 different media, 1 offsite).
- Automate backups and monitor for success.



Back Up Today
Recover Tomorrow

4. Restore Testing

- Regularly test your backups.
- Verify that backups are complete and usable.
- Test in a non-production environment.
- Document the restore process and required time.



A Backup You Can Restore
Is Real Protection

5. RTO (Recovery Time Objective)

- The maximum acceptable time to restore a service after a failure.
- Defined per system based on business needs.
- Lower RTO usually requires more investment.



RTO

How Fast Must We Recover?

6. RPO (Recovery Point Objective)

- The maximum acceptable amount of data loss.
- Defined per system based on business needs.
- Lower RPO requires more frequent backups or real-time replication.



RPO

How Much Data Can We Lose?

7. Multi-AZ Architecture

- Deploy across multiple Availability Zones (AZs).
- Survive the failure of a single AZ automatically.
- Use managed services (e.g. RDS Multi-AZ, EKS, ALB).



High Availability Across AZs

8. Multi-Region Architecture

- Deploy in multiple regions for higher resilience.
- Protect against regional disasters.
- Use replication and global DNS (e.g. Route 53).
- More complex, but higher availability.



Multi-Region
Survive Regional Disasters

9. Failover

- Automatically or manually switch to a healthy system.
- Use health checks and monitoring.
- Test failover regularly.
- Ensure data consistency after failover.



Primary Standby
Be Ready to Failover

10. Dependency Failures

- Identify critical dependencies (e.g. databases, APIs, third-party services).
- Monitor dependency health.
- Plan for failures (timeouts, retries, fallbacks).
- Avoid cascading failures.



Plan for Dependency Failures
They Will Happen

11. Circuit Breakers

- Stop calling a failing service after repeated failures.
- Prevent cascading failures across your system.
- Allow the service time to recover.
- Combine with retries and fallbacks.



Open the Circuit
Protect Your System

12. Retries and Exponential Backoff

- Retry failed requests with increasing delays.
- Avoid overwhelming failing services.
- Use jitter to prevent retry storms.
- Set a maximum number of retries.



Retry Smart
Not Harder

13. Graceful Degradation

- Keep core functionality running during partial failures.
- Show fallback responses (e.g. cached data, read-only mode).
- Prioritize critical user journeys.
- Communicate clearly with users.



Degrade Gracefully
Keep Users Informed

14. Disaster Recovery Exercises

- Run regular DR drills.
- Test failover and full restore processes.
- Validate RTO and RPO targets.
- Involve the whole team (ops, security, business).
- Update runbooks based on lessons learned.



Test Today
Be Ready Tomorrow

15. A Backup You Have Never Restored Is an Assumption

- Unverified backups can be corrupt or incomplete.
- Restoring is the only way to prove they work.
- Test regularly in a realistic environment.
- Document results and fix issues.
- Don't assume. Verify.



A Backup You Have Never Restored
Is an Assumption, Not a Recovery Strategy



KEY TAKEAWAYS

- Design for failure, not perfection.
- Use redundancy and multiple AZs or regions.
- Back up and test restores regularly.
- Define and meet RTO and RPO.
- Plan for dependency failures.
- Use circuit breakers, retries, and graceful degradation.
- Run disaster recovery exercises.
- A backup you have never restored is an assumption, not a strategy.
- Resilience builds trust.
- Be prepared, stay online.



VERIQTA

PLAN CHANGES
REDUCE RISK
DEPLOY SAFELY
KEEP SYSTEMS RELIABLE

Production Change Management

Control changes. Reduce risk. Keep systems reliable.

Good change management helps you move fast without breaking things.

PLAN
REVIEW
VERIFY
DEPLOY
MONITOR
LEARN
IMPROVE

1. Change Risk

- Assess the potential impact before making a change.
- Consider what can go wrong, the likelihood, and the business impact.
- Use a risk matrix (low, medium, high).
- High-risk changes need extra review, testing, and a rollback plan.



Understand the Risk
Before You Change

2. Peer Review

- Have another engineer review your changes.
- Catch mistakes early.
- Ensure you follow standards and best practices.
- Use pull requests, code reviews, or change review boards.



Two Sets of Eyes
Prevent Outages

3. Infrastructure Review

- Review infrastructure changes (e.g. Terraform, CloudFormation, Kubernetes manifests).
- Check for security, cost, scalability, and reliability impact.
- Validate against architecture standards.



Review Infrastructure
Not Just Application Code

4. Maintenance Windows

- Schedule changes during approved maintenance windows.
- Choose a time with the lowest user impact.
- Communicate the window to all stakeholders.
- Have support engineers available.



Pick the Right Time
Minimize User Impact

5. Change Records

- Document every change (before, during, and after).
- Include what was changed, why, who approved it, and the result.
- Keep records in a change management system or ticketing tool.



Good Records Help
With Audits and Troubleshooting

6. Deployment Verification

- Verify the change worked as expected.
- Check key functionality, metrics, and logs.
- Confirm there are no unintended side effects.
- Use automated and manual validation.



Don't Assume It Worked
Verify It

7. Feature Flags

- Use feature flags to release features gradually.
- Enable or disable features without redeploying.
- Reduce risk by testing in production with a small percentage of users.



Release Safely
Control the Rollout

8. Rollback Plans

- Have a clear rollback plan before making the change.
- Know how to quickly revert to the previous state.
- Test the rollback process regularly.
- Keep backups and previous versions available.



Plan for Rollback
Be Ready to Revert

9. Database Changes

- Treat database changes as high risk.
- Test changes in a staging environment first.
- Use backward-compatible changes when possible.
- Have a rollback plan (e.g. scripts, snapshots).



Database Changes Need
Extra Care

10. Emergency Changes

- Sometimes changes are needed urgently.
- Follow a simplified but controlled process.
- Document the change clearly afterwards.
- Do a full review and post-incident analysis.



Move Fast but Stay Controlled
Document Everything

11. Configuration Management

- Use tools like Ansible, Terraform, or Kubernetes to manage configurations.
- Keep configurations in version control.
- Apply changes consistently across environments.
- Avoid manual changes on production systems.



Manage Configurations
Keep It Consistent

12. Drift Detection

- Detect when actual infrastructure differs from the expected state.
- Use tools like Terraform plan, AWS Config, or Kubernetes drift detection.
- Investigate and fix unexpected changes.



Detect Drift Early
Keep Systems in the Expected State

13. Auditability

- Ensure all changes are traceable.
- Log who made the change, when, and what was changed.
- Keep logs in a centralized system (e.g. CloudTrail, SIEM).
- Be ready for internal and external audits.



Trace Changes
Build Trust

14. Reduce Blast Radius

- Limit the impact of changes by using smaller deployments (e.g. canary, blue-green).
- Use feature flags and scoped rollouts.
- Isolate changes to specific services or environments.
- Design systems to fail safely.



Smaller Changes
Safer Systems

15. Senior Engineers Think About Recovery Before Execution

- Always ask: How do we recover if this goes wrong?
- Have rollback plans, backups, and recovery procedures ready before making the change.
- Plan for failure, not just success.
- This mindset separates senior engineers from junior engineers.



Plan Recovery First
Execute with Confidence



KEY TAKEAWAYS

- Assess risk before every change.
- Use peer and infrastructure reviews.
- Schedule maintenance windows.
- Keep detailed change records.
- Verify deployments.
- Use feature flags.
- Have rollback plans.
- Treat database changes carefully.
- Follow a controlled process for emergency changes.
- Detect and fix configuration drift.
- Ensure auditability.
- Reduce blast radius.
- Think about recovery before execution.



Cost, FinOps, and Engineering Tradeoffs

Good engineers build systems that work. Great engineers build systems that are reliable, scalable, and cost efficient.

MEASURE
OPTIMIZE
ELIMINATE
RIGHTSIZE
INVEST
GROW

1. Cloud Cost Fundamentals

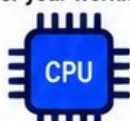
- Understand how cloud providers price their services (e.g. AWS, Azure, GCP).
- Know the main cost drivers: compute, storage, network, databases, and managed services.
- Use cost management tools (e.g. AWS Cost Explorer, Azure Cost Management).
- Set budgets and alerts.



Know What You Are Paying For

2. Compute Utilization

- Monitor CPU and memory utilization.
- Identify over-provisioned instances.
- Use auto scaling and rightsizing.
- Choose the right instance type for your workload.



Higher Utilization
Lower Cost

3. Storage Cost

- Understand storage types (e.g. SSD, HDD, object storage).
- Use the right storage class for your use case (e.g. S3 Standard vs Glacier).
- Remove unused and old data.
- Consider lifecycle policies.



Store Smart
Pay Less

4. Network and Data-Transfer Cost

- Data transfer between regions, and out to the internet can be expensive.
- Keep resources in the same region and VPC when possible.
- Use CDNs for global delivery.
- Monitor ingress and egress traffic.
- Optimize data transfer architecture.



Data Moves Cost Money
Design Efficiently

5. Kubernetes Cost

- Understand the cost of worker nodes, control plane, and managed services (e.g. EKS, AKS, GKE).
- Optimize resource requests and limits.
- Use cluster autoscaling.
- Remove unused namespaces and workloads.
- Monitor with tools like KubeCost.



Efficient Clusters
Lower Bills

6. Idle Resources

- Find and remove unused instances, volumes, load balancers, and IP addresses.
- Stop non-production environments when not in use.
- Automate cleanup with scripts or policies.



Idle Resources
Waste Money

7. Rightsizing

- Choose the right instance size for your actual usage.
- Monitor and analyze usage patterns.
- Downsize or upgrade based on real data.
- Rightsize continuously, as workloads change.



Right Size
Right Cost

8. Reserved Capacity and Commitments

- Use Reserved Instances (RIs), Savings Plans, or Committed Use Discounts.
- Best for predictable and long-term workloads.
- Compare on-demand vs reserved pricing.
- Regularly review your commitments.



Commit Wisely
Save Significantly

9. Autoscaling

- Scale up during demand and scale down when not needed.
- Use metrics (CPU, memory, requests) to trigger scaling.
- Avoid over-provisioning.
- Combine with rightsizing and cost monitoring.



Scale with Demand
Not Emotion

10. Observability Cost

- Logs, metrics, and traces can be expensive at scale.
- Set retention periods.
- Filter and sample data (e.g. reduce log verbosity).
- Choose the right tier for long-term storage.
- Monitor observability spend.



Observe Smart
Control the Cost

11. Cost Attribution

- Tag your resources properly (e.g. team, environment, project, cost center).
- Use cost allocation reports.
- Make cost visible to engineering teams.
- Hold teams accountable for their usage.



Know Who Uses What
Drive Accountability

12. Reliability vs Cost

- Higher reliability (e.g. multi-AZ, multi-region) increases cost.
- Understand the tradeoffs.
- Choose the right level of reliability for your business needs.
- Avoid over-engineering.



Find the Right Balance
For Your Business

13. Performance vs Cost

- Higher performance often means higher cost.
- Choose the right balance for your use case.
- Use caching, CDNs, and right-sized infrastructure.
- Measure the performance improvement against the cost.



Better Performance
At the Right Cost

14. Build vs Buy

- Decide when to build in-house and when to use managed services.
- Managed services reduce operational overhead but can be more expensive.
- Building in-house can be cheaper but requires maintenance and expertise.
- Consider total cost of ownership (TCO).



Build When It Makes Sense
Buy When It Adds Value

15. Senior Engineers Understand The Financial Consequence

- Every architecture decision has a cost impact.
- Consider long-term operational costs, not just initial cost.
- Think about scale, maintenance, and people cost.
- Design systems that are reliable, efficient, and sustainable.



Good Architecture
Controls Cost and Risk

KEY TAKEAWAYS

- Understand your cloud bill.
- Remove idle resources.
- Rightsize and autoscale.
- Use reserved capacity for predictable workloads.
- Monitor observability costs.
- Tag and allocate costs.
- Balance reliability, performance, and cost.
- Choose build vs buy wisely.
- Every design decision has a financial consequence.



What Actually Makes a Senior DevOps Engineer

It's not just about tools. It's about how you think, solve problems, make decisions, and take responsibility.

SYSTEMS
THINKING
BETTER
DECISIONS
STRONGER
ENGINEERS
BRIGHTER
TOMORROW

1. Understand Systems Beyond Individual Tools

- See how all components fit together (applications, infrastructure, data, people).
- Understand dependencies and interactions.
- Think in systems, not tools.



Big Picture Thinking
Creates Better Solutions

2. Debug Unfamiliar Failures

- Stay calm and methodical even when it's a tool you haven't used before.
- Use logs, metrics, traces, and documentation.
- Apply first principles and structured troubleshooting.



It's Not About Knowing
Everything, But Knowing
How to Find Answers

3. Read Architecture Critically

- Analyze architecture diagrams and designs.
- Identify weaknesses, single points of failure, and missing components.
- Ask the right questions before approval.



Good Engineers Read.
Senior Engineers Question.

4. Predict Blast Radius

- Understand what a change can affect (services, users, regions, data).
- Consider direct and indirect impacts.
- Always think: "What else could break?"



Think Beyond
The Immediate Change

5. Challenge Unsafe Changes

- Speak up when something is risky.
- Ask for evidence, testing, rollback plans, and approvals.
- It's not about saying no, it's about keeping production safe.



A Senior Engineer
Protects the Business

6. Design for Failure

- Build systems that can handle failures.
- Use redundancy, retries, timeouts, and graceful degradation.
- Assume things will fail, and design accordingly.



Failure Will Happen.
Be Ready.

7. Protect Production

- Prioritize stability, availability, and data safety.
- Follow change management and best practices.
- Verify, monitor, and have a rollback plan.



Production Trust
Is Earned, Not Given

8. Make Security-Conscious Decisions

- Consider security in every decision.
- Apply least privilege, secure configurations, and secrets management.
- Balance security with usability and performance.



Security Is Everyone's Job
(Especially Yours)

9. Balance Reliability, Speed, Complexity, and Cost

- Make tradeoffs based on real business needs.
- Avoid over-engineering.
- Find the right balance for your context.



There Is No Perfect Solution.
Only Better Tradeoffs.

10. Communicate During Incidents

- Share clear, timely, and accurate updates.
- Translate technical issues for non-technical stakeholders.
- Keep people informed and aligned.



Good Communication
Reduces Panic and Builds Trust

11. Mentor Junior Engineers

- Share knowledge and experience.
- Give constructive feedback.
- Help others grow without doing everything for them.



Senior Engineers Lift Others

12. Document Important Decisions

- Write down architecture decisions, runbooks, and postmortems.
- Help future engineers understand the why.
- Good documentation reduces repeated mistakes.



If It's Not Documented,
It's a Risk

13. Know When Automation Is Appropriate

- Automate repetitive, error-prone tasks.
- Don't automate everything.
- Consider maintenance cost and complexity.



Automate When It Adds Value

14. Know When Simplicity Is Better

- Choose simple solutions when they solve the problem.
- Avoid unnecessary complexity.
- Simple systems are easier to operate and more reliable.



Simple Often Wins

15. Admit Uncertainty and Gather Evidence

- It's okay to say "I don't know."
- Be curious and seek evidence.
- Make decisions based on facts, not assumptions.



Confidence + Evidence
Leads to Better Decisions

16. Own Outcomes, Not Merely Tickets

- Focus on solving the real problem, not just closing a ticket.
- Take responsibility for the outcome.
- Think about long-term impact, not short-term convenience.



Senior Engineers Take Ownership

17. THE BIGGER QUESTION

Junior engineers often ask:

"What command
should I run?"



Senior engineers increasingly ask:

"What is the system doing,
what evidence proves it, what could
this change affect, and how will we
recover if I am wrong?"



Different mindset. Bigger impact.

KEY TAKEAWAYS

- Think in systems, not tools.
- Ask better questions.
- Be reliable, security-conscious, and intentional.
- Balance speed, reliability, complexity, and cost.
- Communicate, document, and mentor.
- Design for failure and always plan recovery.
- Own outcomes.
- Senior engineers make the organization safer, faster, and stronger.

LEARN
PRACTICE
BUILD
GROW

Better
Engineers
Brighter
Tomorrow